Working With Lists

In the previous lesson, we introduced lists as ordered and changeable collections used to store multiple values under one variable name. However, knowing how to create a list is only the beginning. To actually use a list in a program, you'll need to know how to retrieve values from it, change its contents, and manage the items stored inside it.

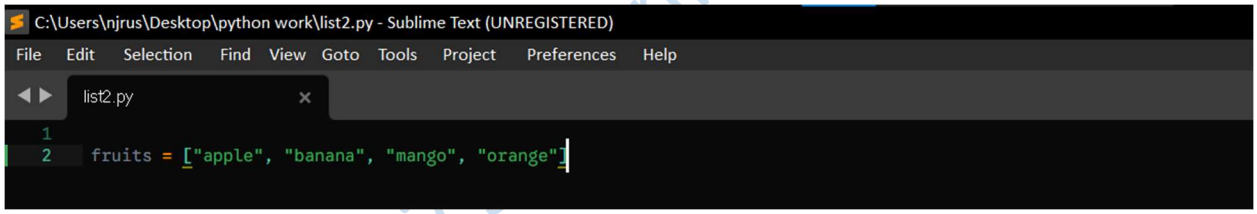
Therefore, we will have to focus on the following 5 important skills:

- Accessing Items in a List
- Indexing in Lists
- Changing Items in a List
- Adding & Removing Items to & from Lists
- Some Useful List Methods

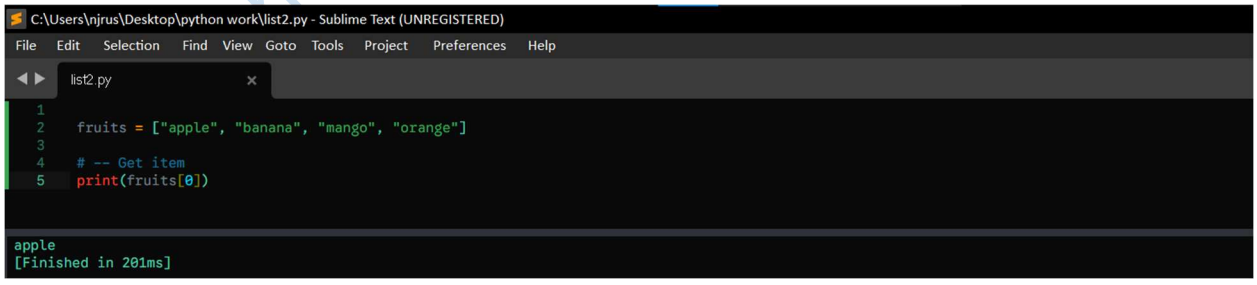
1. Accessing an Item in a List

Accessing an item means **retrieving a specific value** that is stored inside a list. Since a list can contain many values, Python needs a way to know exactly which value we want.

Consider this list:



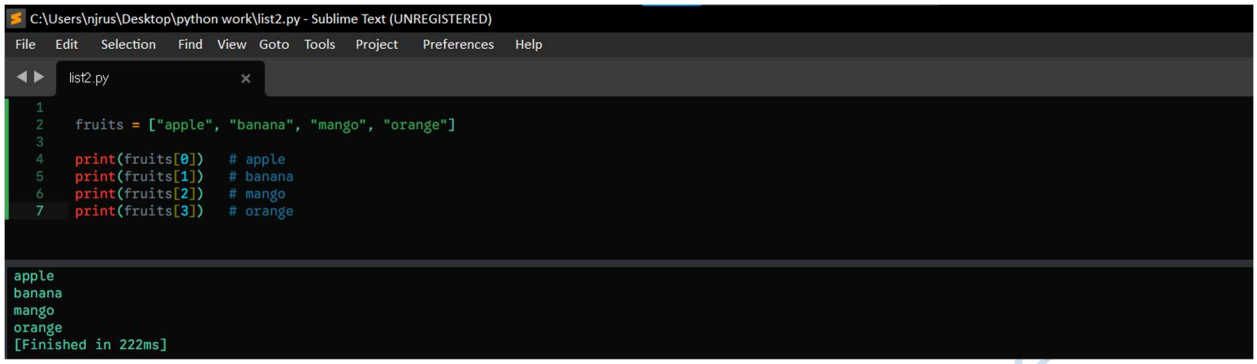
If we want only one item from this list, **we use square brackets after the list name** and **provide the position of the item** we want.



Here, fruits[0] means that you want the item positioned at 0. Since **apple** is the item at position **0,** python prints the value "apple".

Note: The number inside the square brackets is called an index. The index tells Python which item to retrieve.

Accessing Different Items

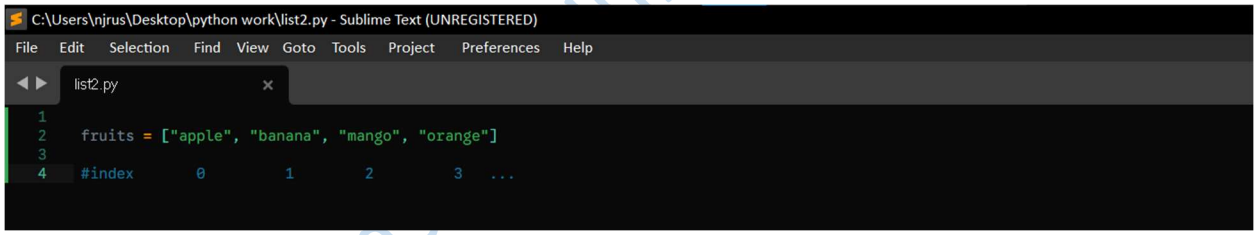


NOTE: Square brackets are used for both creating a list and accessing an item, but they are used differently. When creating a list, the square brackets surround all the items. When accessing an item, the square brackets come after the list name and contain an index.

2. Indexing in Lists

Indexing is the system Python uses to identify the position of each item in a list as each item has an index number.

Now, one of the most important things to remember is that Python uses **zero-based indexing**. This means the first item in a list **has index 0, not index 1.**



Therefore:

fruits[0] = "apple"

fruits[1] = "banana"

fruits[2] = "mango"

fruits[3] = "orange"

.

.

.

and so on

Each Item in a list has an index.

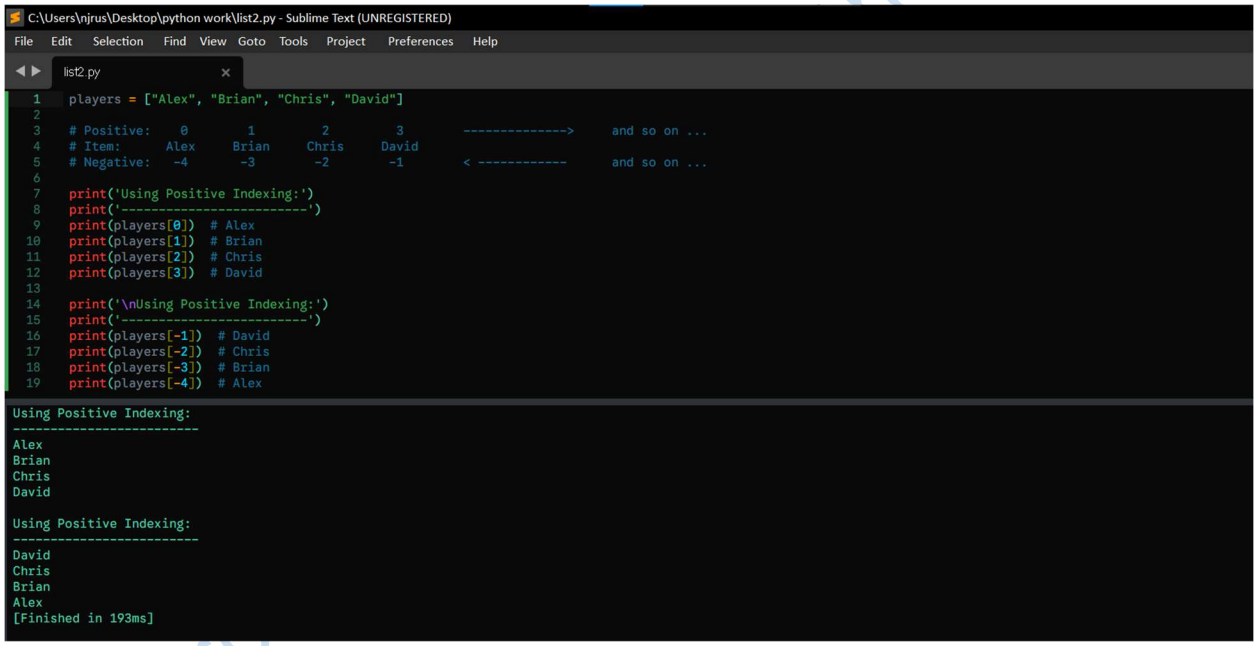
A common mistake many beginners make is thinking that the first item in a list is at index 1. In Python, indexing starts at 0, meaning the first item is at index 0. Avoiding this mistake is important, as using the wrong index can cause errors when accessing data in larger programs.

There are two types of indexing we need to look at:

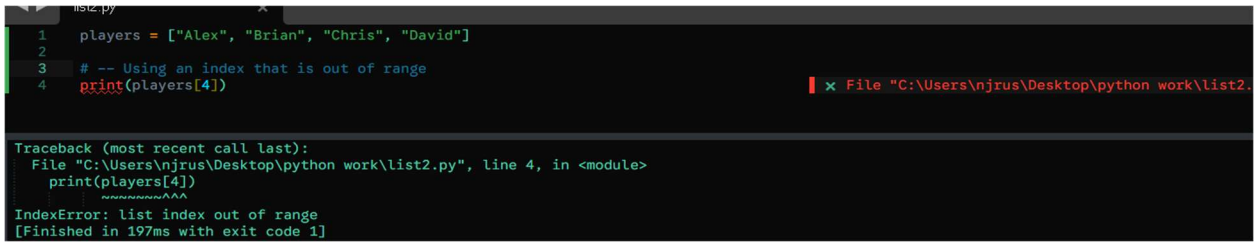
- **Positive Indexing**
  Positive indexing counts from the beginning of the list, starting at 0.

- **Negative Indexing**
  Python also allows negative indexing. Negative indexes count from the end of the list. The last item has index **-1**(**not 0**), the second-last item has index **-2**, and so on. Negative indexing is particularly useful when you want an item near the end of a list without first calculating the list length.

See below the differences between the two:



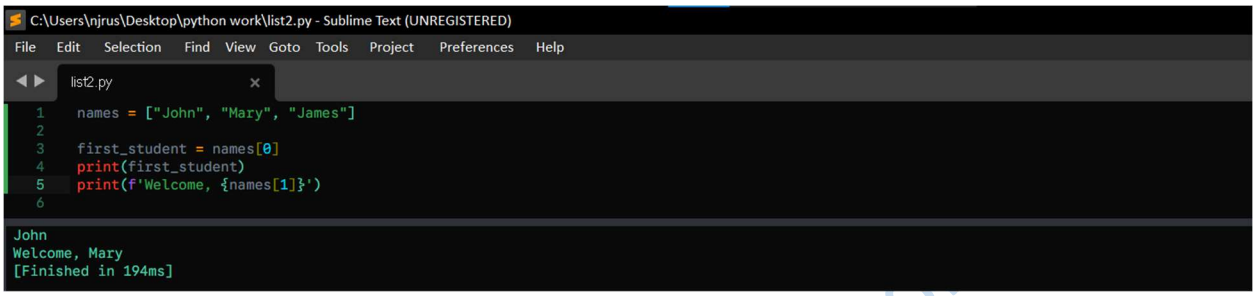
Note: An index must refer a real position in a list. For example, if a list contains five items, its valid positive indexes are 0, 1, 2, 3 & 4 while the valid negative indexes are -1, -2, -3, -4 & -5. If you try using an index that does not refer a real position, python will raise an error. See below.



Python raises an IndexError which means that you tried retrieving an item in a position that is out of range. To avoid such errors, you'll need to use valid indexes. That is, Indexes that refer real positions in the particular list you are trying to retrieve items from.

Accessing a Value and Using It

The value returned from a list can be used just like any other value in Python.



The accessed value can be printed, stored in another variable, compared in a condition, passed to a function **(we will cover this in Functions)**, or used in another expression.

Try Yourself

Create a list called animals containing five animals.

- **Print the first item.**
- **Print the third item in the list.**
- **Print the last item using negative indexing.**
- **Print the second-last item using negative indexing.**
- **Predict the valid positive indexes for a list containing six items.**

From Basics to Junior Python